ЛЕКЦИЯ

Работа приложения, API, сервер и инфраструктура

Путь одного действия через браузер, обработку и хранение

Преподаватель



Цель и результаты занятия

Путь запроса, согласованный API и схема запуска приложения

01

1

Проследить HTTP-запрос от браузера до подтверждённого сохранения.

02

2

Объяснить проверку пользователя, прав и текущего состояния.

03

3

Разобрать конкуренцию запросов и условия безопасного выпуска.

ВОПРОСЫ ТЕМЫ

Вопросы темы

01 Части приложения

02 HTTP и JSON

03 Договор API

04 Обработка на сервере

05 Работа с базой

06 Путь заявки № 104

07 Размещение приложения

08 Выпуск версии

Материалы к занятиям



Ссылка на материалы

https://disk.yandex.ru/d/P40mjNJWYJAW9g

Отсканируйте QR-код или откройте ссылку.

Основная литература

Муромцев В. В. Информационные технологии в профессиональной деятельности : учебник и практикум / В. В. Муромцев, А. В. Муромцева. — Москва, Вологда : Инфра-Инженерия, 2023. — 383 с. — ISBN 978-5-9729-1299-5.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник

Мейер Б. Основы объектно-ориентированного проектирования : учебник для СПО / Б. Мейер. — 2-е изд. — Саратов : Профобразование, 2024. — 751 с. — ISBN 978-5-4488-1002-2.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник

Дополнительная литература

Крынецкая Г. С. Вычислительные машины, сети и системы : учебник / Г. С. Крынецкая. — Москва : Издательский Дом МИСиС, 2023. — 614 с. — ISBN 978-5-907560-73-4.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник

Алексахин А. Н. Программное обеспечение управления проектами : учебник / А. Н. Алексахин, И. Ю. Владыко, Е. В. Сибирская ; под редакцией Е. В. Сибирской. — Москва : Университет «Синергия», 2025. — 144 с. — ISBN 978-5-4257-0670-6.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник



ВОПРОС 1

Части приложения

Показать состояние и передать пользовательские данные

Ответственность сервера

Проверки предшествуют изменению данных

Ответственность браузера

Подготовка
Получить выбранный проект и мотивацию

Отправка
Передать согласованное обращение

Представление
Показать ожидание, результат или неопределённость

Ответственность сервера

Установить пользователя
Получить Анну из действующего сеанса

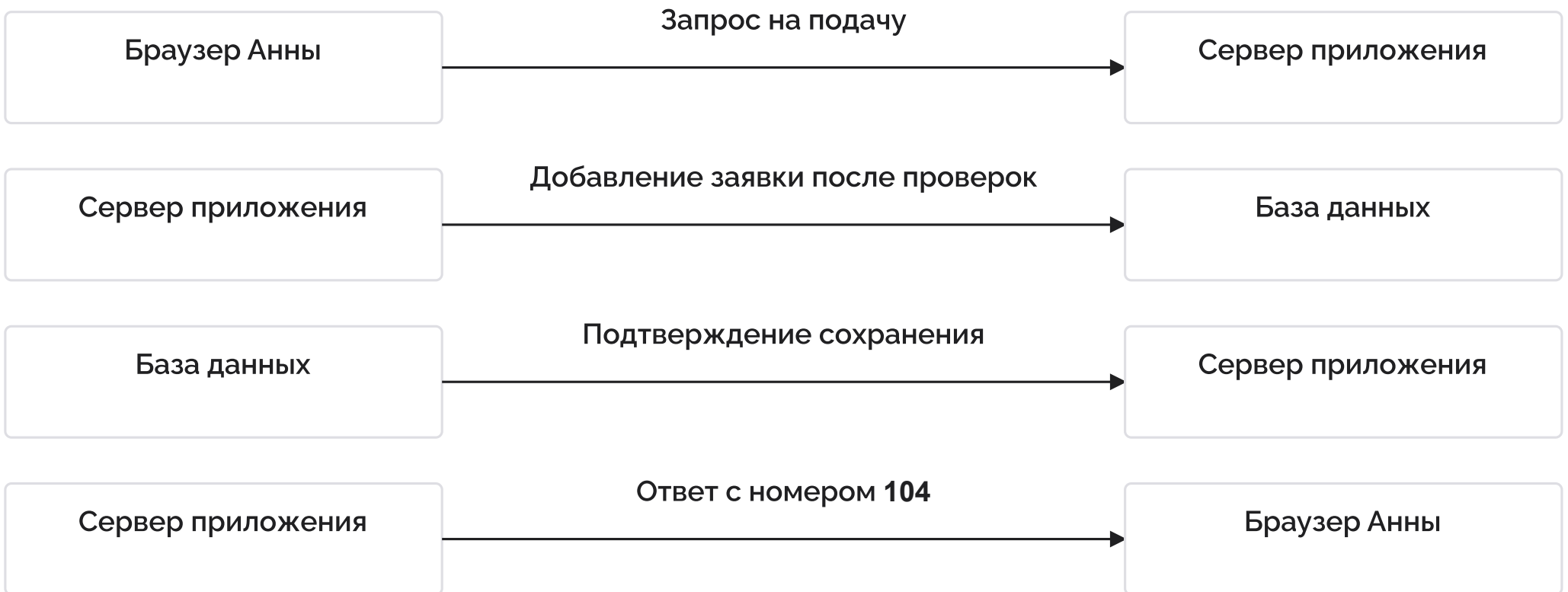
Проверить
Формат, роль, проект, текст и повтор

Сохранить
Создать согласованную запись

Ответить
Сообщить подтверждённый результат

Роль базы в обмене

Сохранённое состояние переживает завершение запроса



ВОПРОС 2

HTTP и JSON

Правила запроса и ответа между участниками веб-обмена

Адрес запроса

Путь обозначает ресурс внутри приложения

Запишите

HTTP — протокол обмена сообщениями запроса и ответа между клиентом и сервером.

HTTP

```
GET /api/projects
Accept: application/json
```

Упрощённая запись запроса каталога. Версия протокола и технические заголовки опущены.

Адрес запроса

Часть адреса	Учебный пример	Смысл
Схема	https	Защищённое соединение
Имя узла	projects.example.test	Условный адрес приложения
Путь	/api/projects/17	Проект 17
Полный пример	https://projects.example.test/api/projects/17	Не является действующим сервисом

Метод запроса

GET, POST и PATCH выражают разные действия

Метод	В нашем договоре	Пример
GET	Получить сведения	/api/projects
POST	Создать заявку	/api/applications
PATCH	Частично изменить заявку	/api/applications/104

Заголовки

Служебные сведения помогают интерпретировать сообщение

```
Content-Type: application/json
Accept: application/json
```

Content-Type описывает передаваемое содержимое. Accept задаёт предпочтительный формат ответа.

Тело запроса

Передаются выбранный проект и мотивация

```
{
  "project_id": 17,
  "motivation": "Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба."
}
```

Тело создания заявки. student_id и status клиент не задаёт.

Ответ сервера

Код и содержимое описывают результат операции

```
201 Created
Location: /api/applications/104
Content-Type: application/json

{"id":104,"project_id":17,
 "student_id":12,"status":"pending"}
```

Сокращённый пример успешного ответа после сохранения.

Формат JSON

Имена полей и значения образуют структуру данных

Фрагмент	Тип	Смысл
"project_id": 17	Число	Идентификатор выбранного проекта
"motivation": "..."	Строка	Текст из формы
"is_open": true	Логическое значение	Признак открытого набора

Группы результатов

Успех, отказ по запросу и сбой обработки различаются

Код	Значение в нашем договоре
200 / 201	Успешное чтение или создание
401 / 403	Вход не подтверждён или действие запрещено
409	Конфликт: повтор, закрытый проект, недопустимый переход
422	Формат понятен, содержимое не соответствует правилам
5xx	Сервер не смог нормально выполнить обработку

ВОПРОС 3

Договор API

Операция включает путь, данные, права и результат

Получение каталога

GET /api/projects возвращает проекты

Запишите

API — согласованный интерфейс взаимодействия программных компонентов.

API как договор

Обращение
Метод, путь и допустимое содержимое

Условия
Кто вызывает и какие правила выполнены

Результат
Успешное представление или согласованная ошибка

Получение каталога

```
GET /api/projects

200 OK
[{"id":17,"title":"Сайт студенческого клуба",
  "teacher_id":7,"is_open":true}]
```

Сокращённое представление каталога, без полного описания каждого проекта.

Получение проекта

GET /api/projects/17 возвращает выбранное описание

```
GET /api/projects/17

200 OK
{"id":17,"title":"Сайт студенческого клуба",
 "description":"Каталог мероприятий и личный кабинет",
 "teacher_id":7,"is_open":true}
```

Данные для карточки выбранного проекта.

Создание заявки

POST /api/applications принимает только проект и мотивацию

```
POST /api/applications
Content-Type: application/json

{
  "project_id": 17,
  "motivation": "Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба."
}
```

Упрощённый запрос. Действующий сеанс подразумевается, технические заголовки опущены.

Получение своих заявок

GET /api/my/applications использует текущего пользователя

```
GET /api/my/applications

200 OK
[{"id":104,"project_id":17,
  "student_id":12,"status":"pending"}]
```

Сокращённый ответ для Анны. Отбор по текущему пользователю выполняет сервер.

Получение заявок проекта

GET /api/projects/17/applications требует права на проект

```
GET /api/projects/17/applications

200 OK
[{"id":104,"project_id":17,"student_id":12,
  "status":"pending"}]
```

Сокращённый ответ Ирине, чей id 7 совпадает с teacher_id проекта.

Решение по заявке

PATCH /api/applications/104 меняет только допустимый статус

1
Пользователь
Установить Ирину по сеансу; не доверять id из тела запроса.

2
Принадлежность
Найти проект заявки; teacher_id должен совпасть с пользователем.

3
Текущее состояние
UPDATE ... WHERE id=$1 AND status='pending' RETURNING id;

4
Результат записи
Одна изменённая строка — решение; ноль — перечитать состояние и вернуть согласованный отказ.

Свой проект; status=pending; новый status=accepted.

1
Пользователь

Установить Ирину по сеансу; не доверять id из тела запроса.

2
Принадлежность

Найти проект заявки; teacher_id должен совпасть с пользователем.

3
Текущее состояние

UPDATE ... WHERE id=$1 AND status='pending' RETURNING id;

4
Результат записи

Одна изменённая строка — решение; ноль — перечитать состояние и вернуть согласованный отказ.

Личность и принадлежность подтверждены.

Условное UPDATE меняет только pending.

Изменена одна строка; после COMMIT можно подтвердить результат.

200 OK · accepted

Свой проект; status=accepted; попытка перевести в rejected.

1
Пользователь

Установить Ирину по сеансу; не доверять id из тела запроса.

2
Принадлежность

Найти проект заявки; teacher_id должен совпасть с пользователем.

3
Текущее состояние

UPDATE ... WHERE id=$1 AND status='pending' RETURNING id;

4
Результат записи

Одна изменённая строка — решение; ноль — перечитать состояние и вернуть согласованный отказ.

Условие status=pending не совпало.

Изменено 0 строк; проверенный актуальный статус уже итоговый.

409 INVALID_TRANSITION · accepted сохранён

Единый формат ошибок

Машинный код причины и текст выполняют разные задачи

```
409 Conflict
Content-Type: application/json

{"code":"APPLICATION_EXISTS",
 "message":"Вы уже подали заявку на этот проект."}
```

Тело ошибки содержит устойчивый **code** и понятное пользователю **message**.

ВОПРОС 4

Обработка на сервере

Упорядоченный алгоритм выполняет одну операцию

Определение пользователя

Сервер связывает запрос с действующим сеансом

Серверный обработчик

```
пользователь = определить_по_сеансу()
проверить_вход_и_роль(пользователь)
данные = разобрать_и_проверить_тело()
проверить_проект_и_мотивацию(данные)
заявка = сохранить_без_повтора(пользователь, данные)
вернуть_подтверждение(заявка)
```

Псевдокод создания заявки: обозначает смысл шагов, а не синтаксис языка.

Определение пользователя

Источник	Можно считать доказательством личности?	Почему
Доверенный контекст сеанса	Да, в принятом учебном допущении	Вход уже проверен системой
Поле student_id в JSON	Нет	Клиент может изменить значение
Имя на картинке формы	Нет	Внешний вид не устанавливает пользователя

Проверка входных данных

Разобранный JSON ещё должен соответствовать договору

Проверка	Корректный вариант	Отказ
Структура	Объект с двумя ожидаемыми полями	Неверное содержимое
project_id	Числовой идентификатор проекта	Отсутствует или неподходящий тип
motivation	Текст после согласованной нормализации	Нет текста или нарушена длина
Лишние поля	Отсутствуют	Не принимаются как полномочие

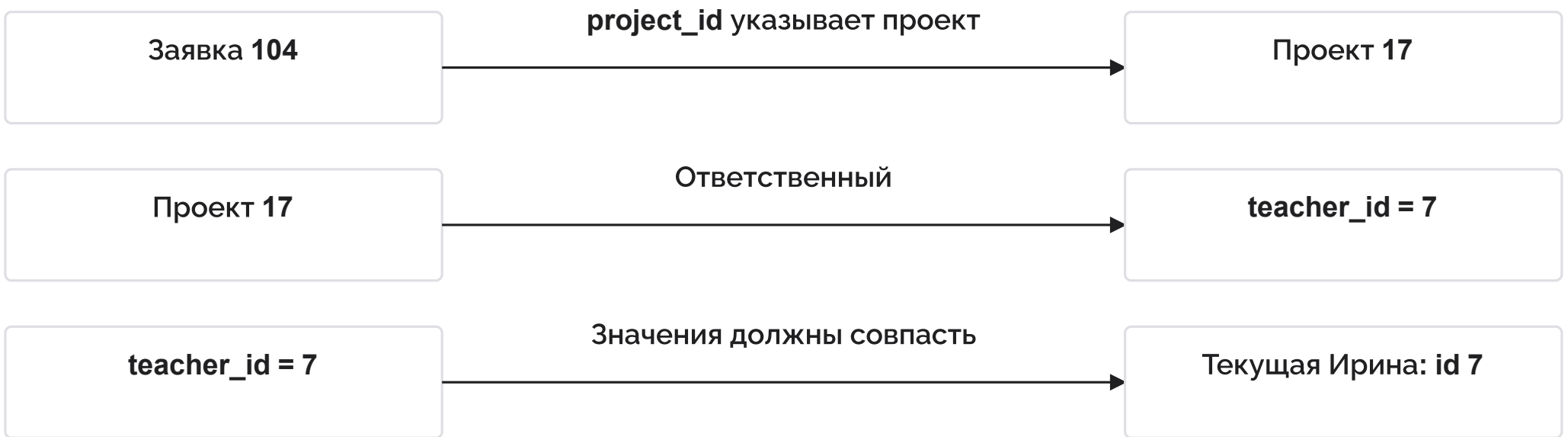
Проверка роли

Установленный пользователь получает только разрешённые действия

Операция	Требуемая роль	Пример отказа
Создание заявки	student	Ирина не подаёт заявку как преподаватель
Решение по заявке	teacher	Анна не принимает собственную заявку
Список проекта	teacher и принадлежность	Роль без связи проекта недостаточна

Принадлежность проекта

Ирина принимает решение по связи teacher_id



Бизнес-правила

Допустимость определяется текущим состоянием предметной области

01

Набор открыт

Проект существует и принимает новые заявки

02

Мотивация допустима

Нормализованный текст имеет 50–1000 символов

03

Повтора нет

Сочетание студента и проекта ещё не занято

04

Переход допустим

Решение применяется только из pending

Обработка отказов

Неправильный путь завершается без недопустимой записи

Причина	Код ответа	Машинная причина
Не подтверждён вход	401	AUTH_REQUIRED
Недостаточно прав	403	FORBIDDEN
Повтор	409	APPLICATION_EXISTS
Проект закрыт	409	PROJECT_CLOSED
Длина мотивации	422	MOTIVATION_LENGTH

Формирование успешного ответа

Подтверждение опирается на результат сохранения

01

Сохранение завершилось

Получен номер созданной заявки

02

Собраны допустимые поля

id, project_id, student_id и status

03

Отправлен ответ

201 Created с сокращённым представлением

ВОПРОС 5

Работа с базой

Запрос выбирает нужные строки и поля

Добавление записи

INSERT создаёт заявку из проверенных значений

Чтение данных

```
SELECT id, title, is_open
FROM projects
WHERE id = $1;
```

Учебный SQL. Параметр $1 получает 17 отдельно от текста запроса.

Добавление записи

```
INSERT INTO applications
  (project_id, student_id, motivation, status, created_at)
VALUES ($1, $2, $3, 'pending', CURRENT_TIMESTAMP)
RETURNING id;
```

Параметры: проект 17, установленный студент 12 и допустимая мотивация. id назначает база.

Связанные данные в SQL

Соединение даёт название проекта для своей заявки

```
SELECT a.id, p.title, a.status
FROM applications AS a
JOIN projects AS p ON p.id = a.project_id
WHERE a.student_id = $1;
```

Параметр $1 берётся из текущего пользователя: для Анны это 12.

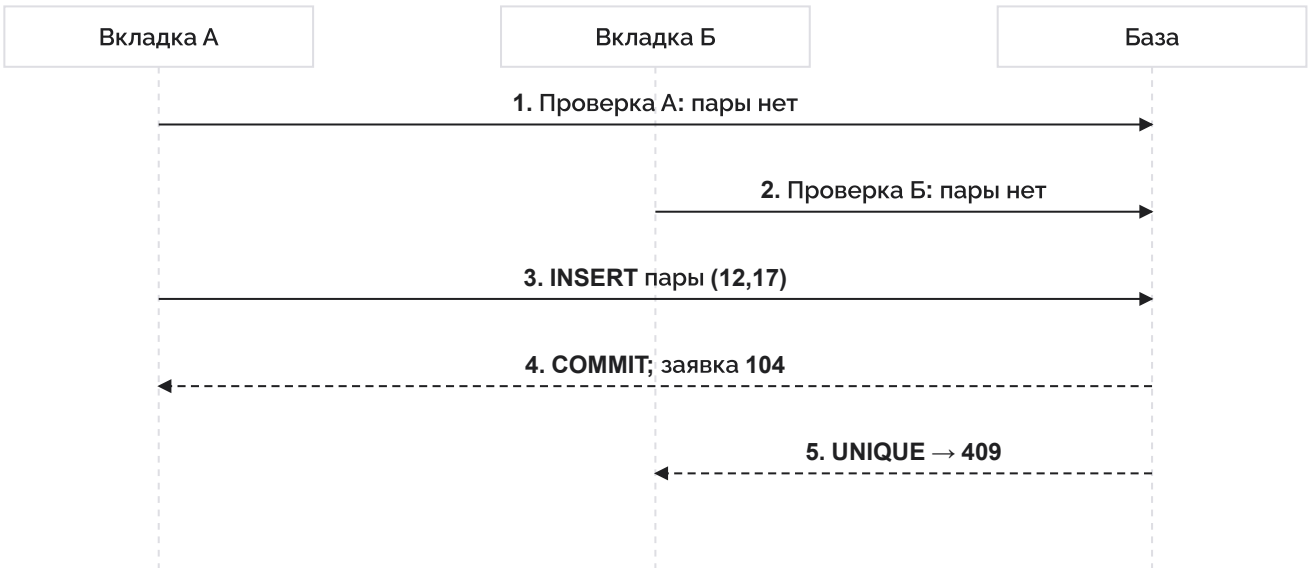
Ограничения базы

Разные ограничения защищают разные свойства данных

Ограничение	Что предотвращает	Чего не доказывает
PRIMARY KEY	Повтор ключа строки	Право пользователя
FOREIGN KEY	Ссылку на отсутствующую запись	Нужную роль пользователя
NOT NULL	Отсутствие обязательного значения	Допустимую длину текста
UNIQUE пары	Повтор студента и проекта	Открытость набора
CHECK статуса	Неизвестное значение	Разрешённый переход

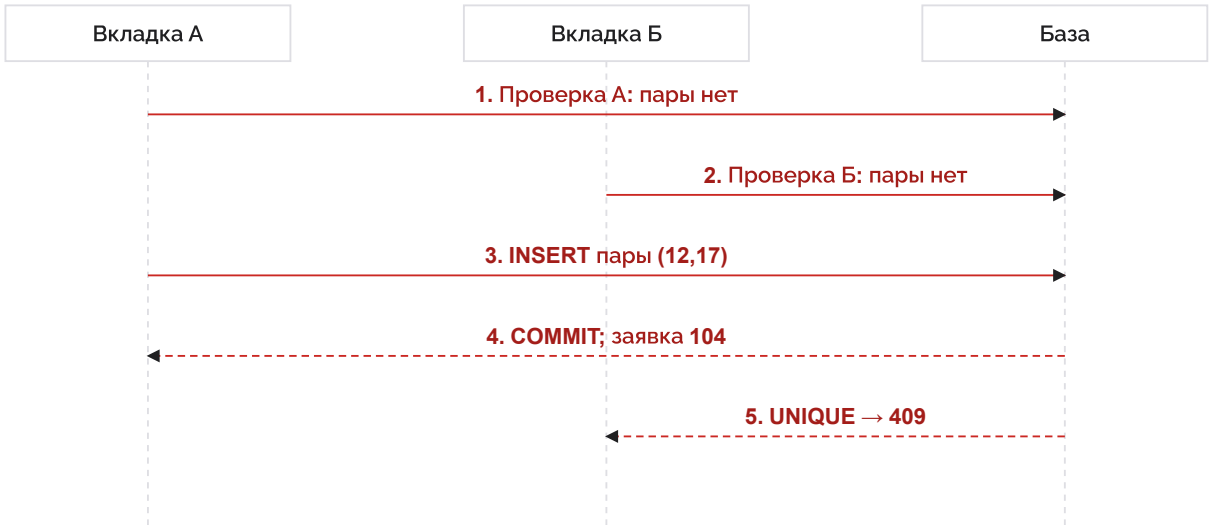
Два одновременных запроса

Оба могут прочитать отсутствие заявки до первой записи



Порядок сообщений условный; вертикальная ось не имеет масштаба времени.

Обе вкладки: student_id=12, project_id=17. Обе успели прочитать отсутствие заявки.



Обе предварительные проверки: пары ещё нет.

Вкладка А: INSERT занимает уникальную пару.

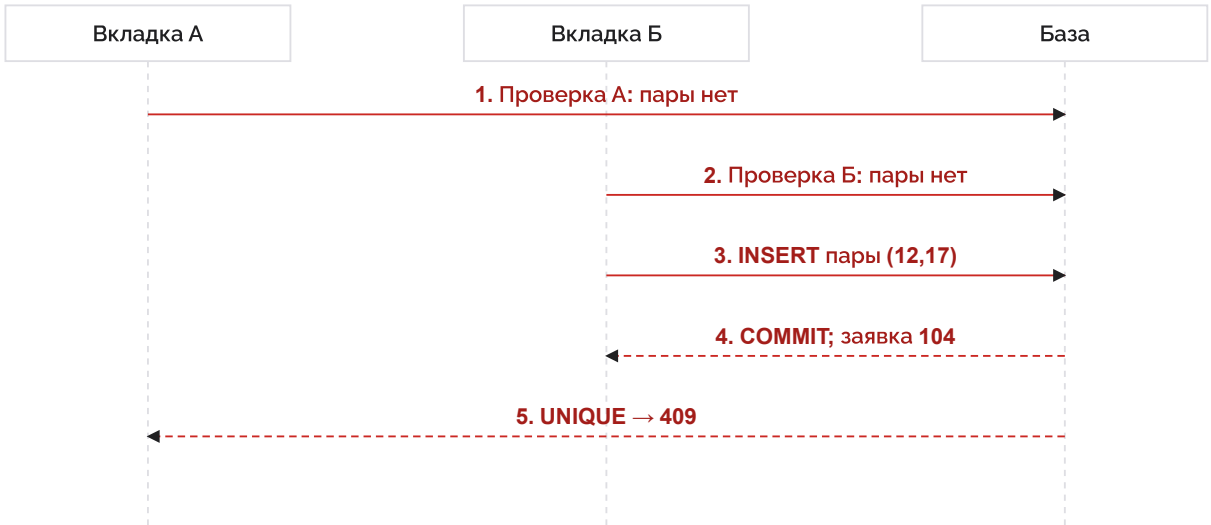
Одна запись; победитель 201; другая попытка 409 APPLICATION_EXISTS

Порядок сообщений условный; вертикальная ось не имеет масштаба времени.

Вкладка А: COMMIT подтверждён.

Другая вкладка: UNIQUE отклоняет повтор той же пары.

Обе вкладки: student_id=12, project_id=17. Обе успели прочитать отсутствие заявки.



Обе предварительные проверки: пары ещё нет.

Вкладка Б: INSERT занимает уникальную пару.

Одна запись; победитель 201; другая попытка 409 APPLICATION_EXISTS

Порядок сообщений условный; вертикальная ось не имеет масштаба времени.

Вкладка Б: COMMIT подтверждён.

Другая вкладка: UNIQUE отклоняет повтор той же пары.

Уникальная пара при конкуренции

Один результат создания, один понятный конфликт

01

Первая фиксация

Пара student_id 12 и project_id 17 занята

02

Следующая попытка

База не допускает вторую такую комбинацию

03

Ответ клиенту

409 APPLICATION_EXISTS и переход к своим заявкам

Транзакция

Связанное изменение завершается целиком или отменяется

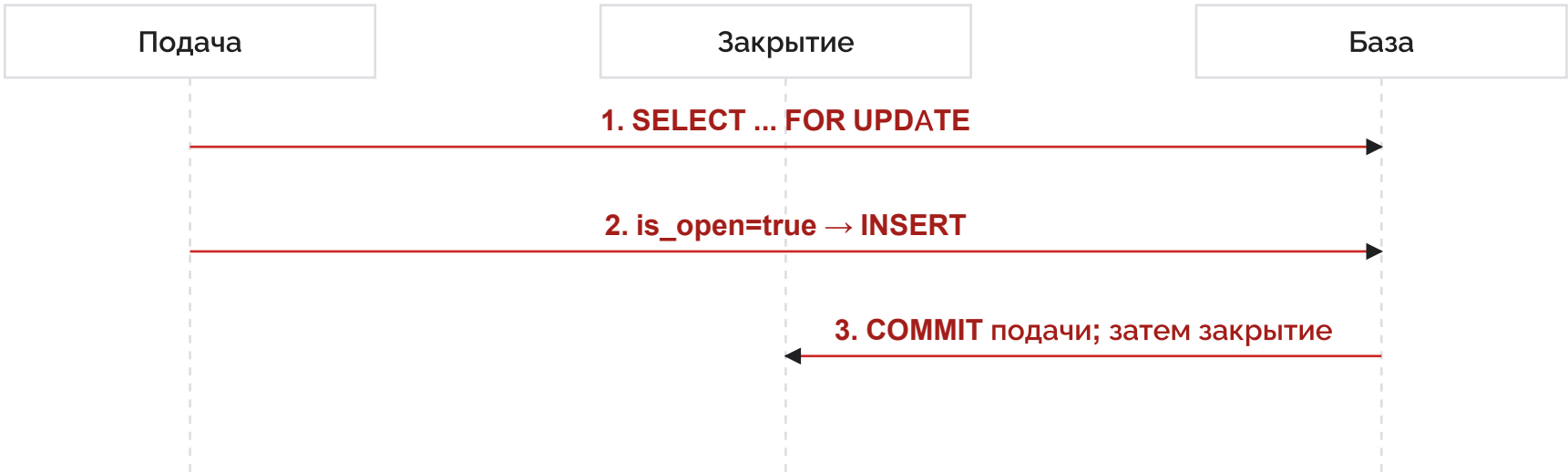
Запишите

Транзакция — единица работы с данными, изменения которой фиксируются вместе либо отменяются.



Оба обработчика используют блокировку одной строки **projects**. Это явно выбранная дисциплина реализации.

Подача первой блокирует projects.id=17; проект ещё открыт.



Подача: SELECT ... FOR UPDATE блокирует строку проекта.

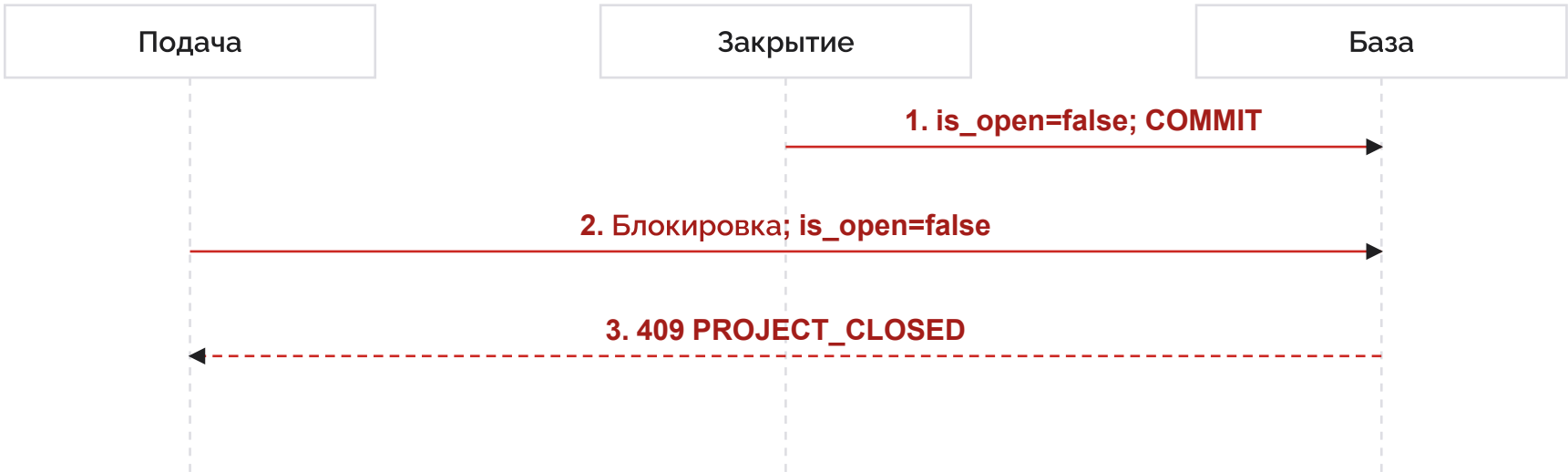
Подача проверяет is_open=true и вставляет заявку.

COMMIT подачи; затем закрытие получает блокировку и меняет is_open=false.

Заявка сохранена до закрытия; будущие подачи запрещены.

Оба обработчика используют блокировку одной строки projects. Это явно выбранная дисциплина реализации.

Закрытие первым блокирует ту же строку и фиксирует is_open=false.



Закрытие: UPDATE projects SET is_open=false; COMMIT.

Подача получает блокировку и читает уже закрытый проект.

Условие is_open ложно: INSERT не выполняется.

409 PROJECT_CLOSED · новой заявки нет

Оба обработчика используют блокировку одной строки projects. Это явно выбранная дисциплина реализации.

Сохранение перед подтверждением

Успешный ответ следует за фиксацией результата

01

Изменение подготовлено

Запись ещё относится к незавершённой работе

02

Изменение зафиксировано

Система получила подтверждение завершения

03

Ответ отправлен

Клиент может показать успешную подачу

ВОПРОС 6

Путь заявки № 104

Перед отправкой выполнены пользовательские условия

Путь заявки: запрос

В сообщение вошли проект 17 и мотивация

Путь заявки: исходная форма

Проектный навигатор Анна Смирнова

Заявка на проект «Сайт студенческого клуба»

Почему вы хотите участвовать?

Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба.

От 50 до 1000 символов

Подать заявку

Путь заявки: запрос

```
POST /api/applications
Content-Type: application/json

{
  "project_id": 17,
  "motivation": "Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба."
}
```

Шаг 2 полного пути. Сеанс Анны подразумевается и не заменяется полем student_id.

Путь заявки: установленный пользователь

Сервер определил Анну с id 12

Проверяемое сведение	Источник	Результат шага
Пользователь	Действующий сеанс	id 12
Роль	Сохранённые сведения пользователя	student
Свободный номер из тела	В договоре отсутствует	Не используется как личность

Путь заявки: проверка проекта

Проект существует и открыт для новой подачи

Сведение проекта 17	Учебное значение	Вывод
Наличие записи	Проект найден	Можно продолжить предметные проверки
is_open	true	Новый приём разрешён
teacher_id	7	Будущая заявка попадёт в область Ирины

Путь заявки: проверка мотивации

Одинаковое правило длины действует на форме и сервере

01

Нормализация

Удалить пробельные символы с концов по общему правилу

02

Подсчёт

Посчитать Unicode code points

03

Диапазон

Принять от 50 до 1000 включительно

04

Результат

Учебная мотивация соответствует условию

Путь заявки: сохранение

Пара 12 и 17 создаётся один раз

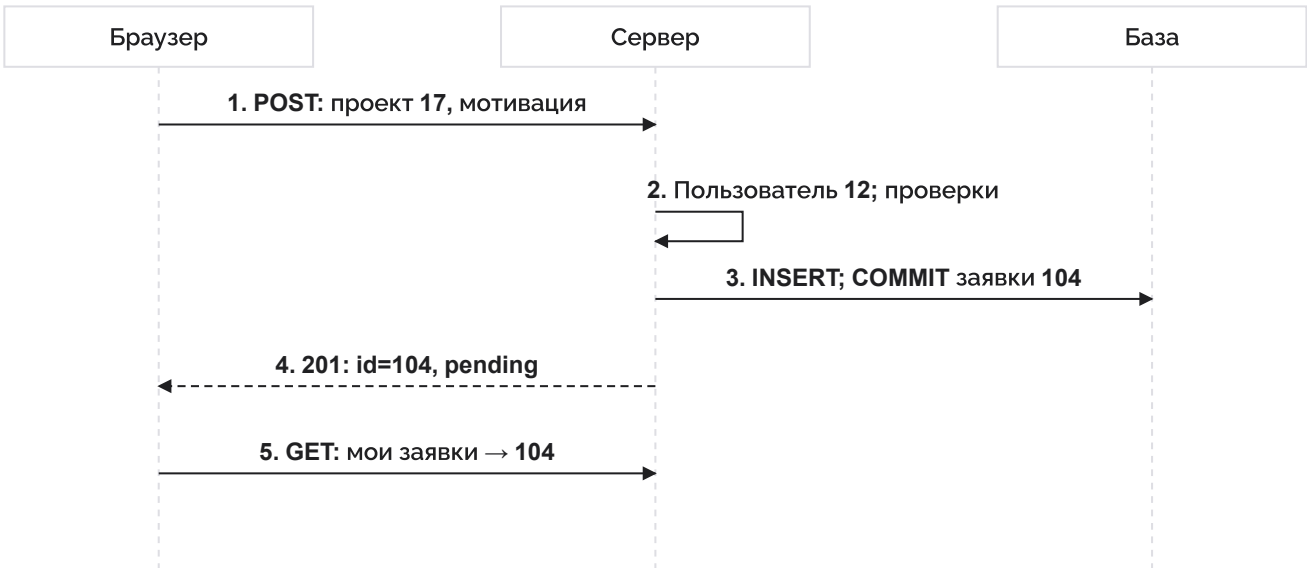


Схема сообщений; порядок сверху вниз, без временной шкалы.

Анна 12; проект 17 открыт; мотивация допустима; пары ещё нет.



POST /api/applications: project_id=17 и motivation.

Сервер устанавливает пользователя 12 и проверяет условия.

INSERT создаёт pending; COMMIT подтверждает сохранение.

Ответ 201: id=104, status=pending.

GET /api/my/applications подтверждает доступную Анне запись.

Одна заявка 104; сохранение предшествует подтверждению.

Схема сообщений; порядок сверху вниз, без временной шкалы.

Путь заявки: успешный ответ

201 сообщает о созданной заявке 104

```
201 Created
Location: /api/applications/104
Content-Type: application/json

{"id":104,"project_id":17,
 "student_id":12,"status":"pending"}
```

Шаг 7. Сокращённый ответ после фиксации записи.

Путь заявки: обновление интерфейса

Подтверждение отражает полученные серверные сведения

Проектный навигатор Анна Смирнова

Заявка сохранена

Браузер получил 201, показал заявку 104, проект 17 и состояние «На рассмотрении».

Заявка сохранена

Заявка № 104 · Сайт студенческого клуба

На рассмотрении

Путь заявки: чтение Анной

Свой список подтверждает устойчивость результата

```
GET /api/my/applications

200 OK
[{"id":104,"project_id":17,
  "student_id":12,"status":"pending"}]
```

Сокращённый результат повторного чтения Анной.

Путь заявки: чтение Ириной

Одна запись доступна преподавателю своего проекта

```
GET /api/projects/17/applications

200 OK
[{"id":104,"project_id":17,
  "student_id":12,"status":"pending"}]
```

Ирина установлена как id 7; teacher_id проекта 17 равен 7.



ВОПРОС 7

Размещение приложения

Одинаковая программа может иметь разные настройки и данные

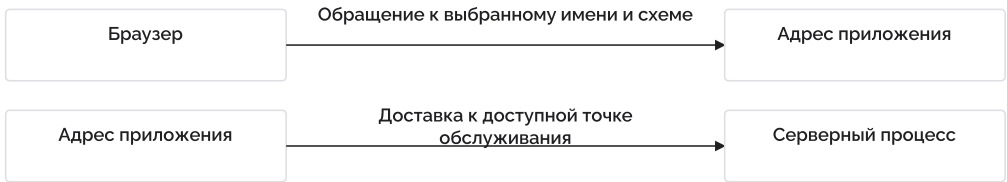
Адрес приложения

Браузеру нужен доступный сетевой узел и путь

Локальная и рабочая среда

Среда	Назначение	Данные
Локальная	Разработка и подготовленные проверки	Учебный набор
Рабочая	Использование ресурса по общему адресу	Рабочие записи
Условие перехода	Настройки согласованы отдельно	Данные не подменяются автоматически

Адрес приложения



Доменное имя

Имя помогает найти нужный узел, но не заменяет приложение

01

Имя

Условный projects.example.test читается человеком

02

Разрешение имени

Система доменных имён помогает определить адрес узла

03

Обращение

Браузер соединяется с точкой обслуживания

HTTPS

Защищённое соединение дополняет правила приложения

Средство	Что обеспечивает	Чего не заменяет
TLS в HTTPS	Защиту обмена и проверку стороны по принятой модели сертификатов	Правильность бизнес-правил
Серверная авторизация	Право Анны или Ирины на действие	Защиту сетевой передачи
Ограничения базы	Согласованность записей	Проверку сетевого сертификата

Процесс сервера

Программа должна оставаться запущенной и доступной

01

Запуск

Рабочая версия начинает принимать обращения

02

Работа

Процесс выполняет обработчики и использует настройки

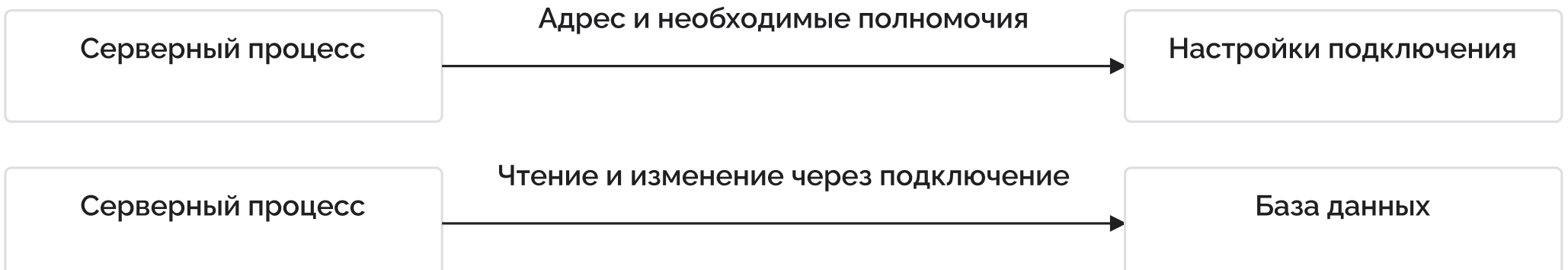
03

Остановка

Новые обращения не получают обычную обработку

Подключение к базе

Обработка использует отдельные параметры доступа к данным



Конфигурация

Параметры среды отделяют от правил и пользовательских данных

Параметр	Учебное обозначение	Назначение
Адрес базы	DB_URL = значение среды	Подключение к нужному хранилищу
Режим	APP_ENV = production	Условия запуска
Адрес приложения	APP_ORIGIN = рабочий адрес	Согласованное обращение
Доступ	Секреты не включены в пример	Не публикуются в клиентском коде

Доступность компонентов

Рабочая страница не доказывает исправность полного пути

Компонент	Признак	Ограничение вывода
Страница интерфейса	Загрузилась оболочка	API может быть недоступен
API каталога	Получен ожидаемый список	Запись может не работать
База	Доступны нужные операции	Права пользователя ещё проверяются
Полный сценарий	Получен согласованный результат	Нужны также отрицательные проверки

ВОПРОС 8

Выпуск версии

Изменение проходит подготовку, размещение и подтверждение

Подготовка настроек выпуска

Рабочая версия получает согласованную конфигурацию

Выпуск версии

Выбор версии
Известно, какой проверенный вариант выпускают

Подготовка
Настройки, совместимость данных и резервирование

Размещение
Обновление и запуск по принятому порядку

Подтверждение
Проверка доступности и ключевого сценария

Подготовка настроек выпуска

Проверяем	Ожидаемое состояние
Среда	Выбрана рабочая, а не локальная
Подключение	Доступна нужная база
Адрес	Совпадает с опубликованным маршрутом
Защищённый обмен	Сертификат и HTTPS подходят адресу
Сведения о версии	Можно установить действующую редакцию

Обновление структуры данных

Новая программа должна понимать существующие записи

01

Изменение схемы

Подготовленная операция меняет структуру хранения

02

Совместимость

Проверяется чтение старых и новых записей

03

Порядок

Обновление данных согласовано с выпуском программы

Резервная копия

Восстановление требует пригодной копии и проверенного порядка

Резервная копия

Шаг	Назначение
Подготовить копию	Сохранить состояние до рискованного изменения
Проверить состав	Убедиться, что нужные данные включены
Проверить восстановление	Установить, что копия действительно пригодна
Учесть время	После копии могли появиться новые записи

Возврат версии и передача в сопровождение

Решение о возврате
Известен признак неудачного выпуска

Проверка совместимости
Предыдущая программа работает с текущими данными

Восстановление
Учитываются изменения схемы и новые записи

Передача
Зафиксированы версия, адреса проверки и порядок разбора

Итоги занятия

Что связывает рассмотренные решения

Связь	Результат
Запрос → полномочие	Сеанс определяет пользователя; тело запроса не доказывает право.
Запись → подтверждение	UNIQUE защищает пару; COMMIT предшествует ответу 201.
Выпуск → проверка	Конфигурация, совместимость схемы и восстановление проверяются отдельно.



Вопросы от аудитории

- Почему номер студента не передаётся при создании?
- Что произойдёт при двух одновременных запросах?
- Когда можно показать подтверждение?